Dynamic Programming

Dynamic Programming

- Dynamic programming เป็นวิธีการแก้ปัญหาแบบหนึ่งที่เราจะแตกปัญหาออกเป็นปัญหาย่อยๆ (ที่แก้ด้วยวิธีเดิม แต่ขนาดปัญหาเล็กลง) จากนั้นจะ**เก็บผลลัพธ์**ของปัญหาย่อยเหล่านี้ เพื่อที่เมื่อมีการหาคำตอบของปัญหาย่อยเหล่านี้อีกจะได้**ไม่ต้องคำนวณใหม่**

- เราจะได้ยินบ่อยๆ ว่า
  - Dynamic programming กับตาราง
  - Bruteforce แบบฉลาด จำไว้

Dynamic Programming

ทั้งนี้คุณสมบัติหลักของปัญหาที่จะแก้ด้วย Dynamic programming ได้คือ

- **Overlapping Subproblems** (มีปัญหาย่อยซ้ำกัน หรือเรียกให้คำนวณฟังก์ชันที่มี parameter เดิมบ่อยๆ)
- **Optimal Substructure** (คำตอบที่ดีที่สุดของปัญหาได้จากคำตอบที่ดีที่สุดของปัญหาย่อย(ปัญหาเดิม แต่มีขนาดที่เล็กกว่า recursive นั่นเอง))

Overlapping Subproblems

- เช่นเดียวกับ Divide and conquer, Dynamic programming นั้นจะรวมเอาคำตอบมากจากปัญหาย่อย

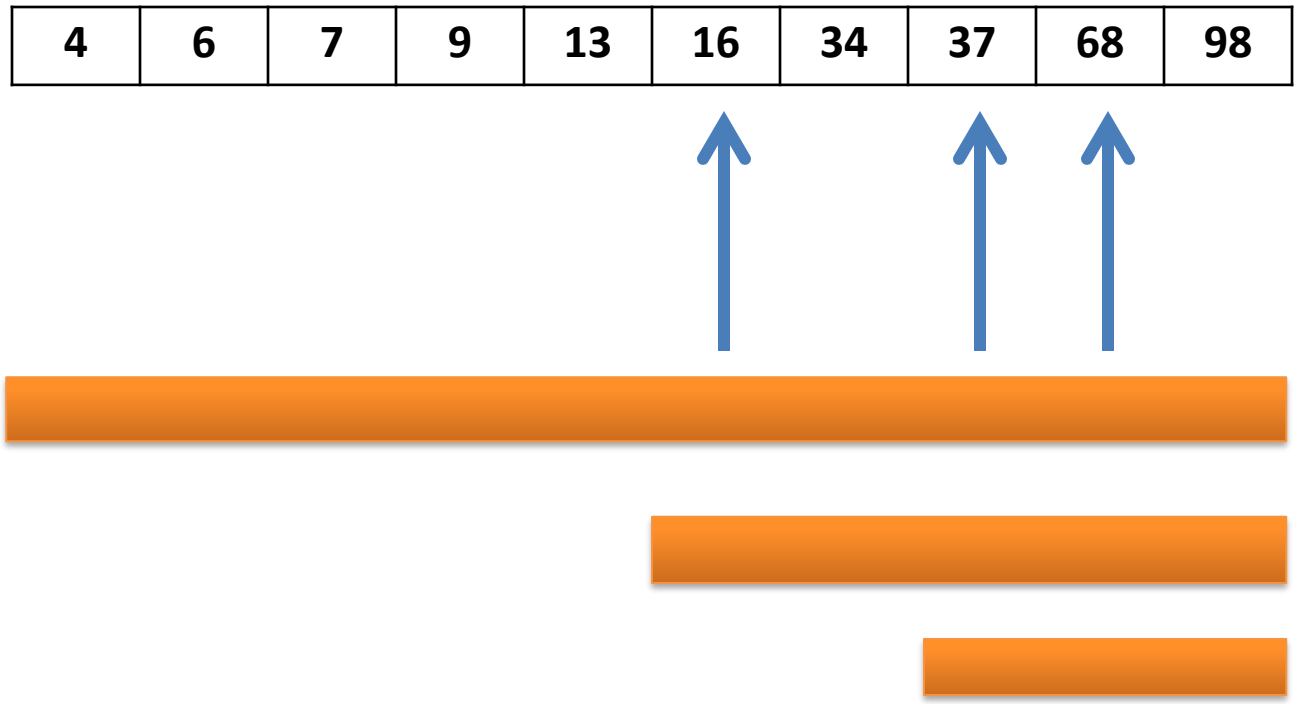
- Dynamic programming จะถูกใช้หลักๆ เมื่อคำตอบของปัญหาย่อยที่**เหมือนกัน**นั้นถูก**คำนวณบ่อยๆ (เรียกด้วย parameter เดียวกัน)** ซึ่งใน Dynamic programming คำตอบที่คำนวณแล้วของปัญหาย่อยจะถูกเก็บไว้ในตารางเพื่อที่ว่าจะได้ไม่ต้องคำนวณใหม่อีก

Overlapping Subproblems

- ดังนั้น Dynamic programming จะ**ไม่**เกิดประโยชน์เท่าไร เมื่อไม่มีปัญหาย่อยที่ซ้ำกันเลย (no common overlapping subproblems) เพราะว่าไม่รู้ว่าจะเก็บใส่ตารางไปทำไม เมื่อไม่ได้ใช้อีก

- ตัวอย่างเช่น binary search ไม่มีปัญหาย่อยที่ซ้ำกัน

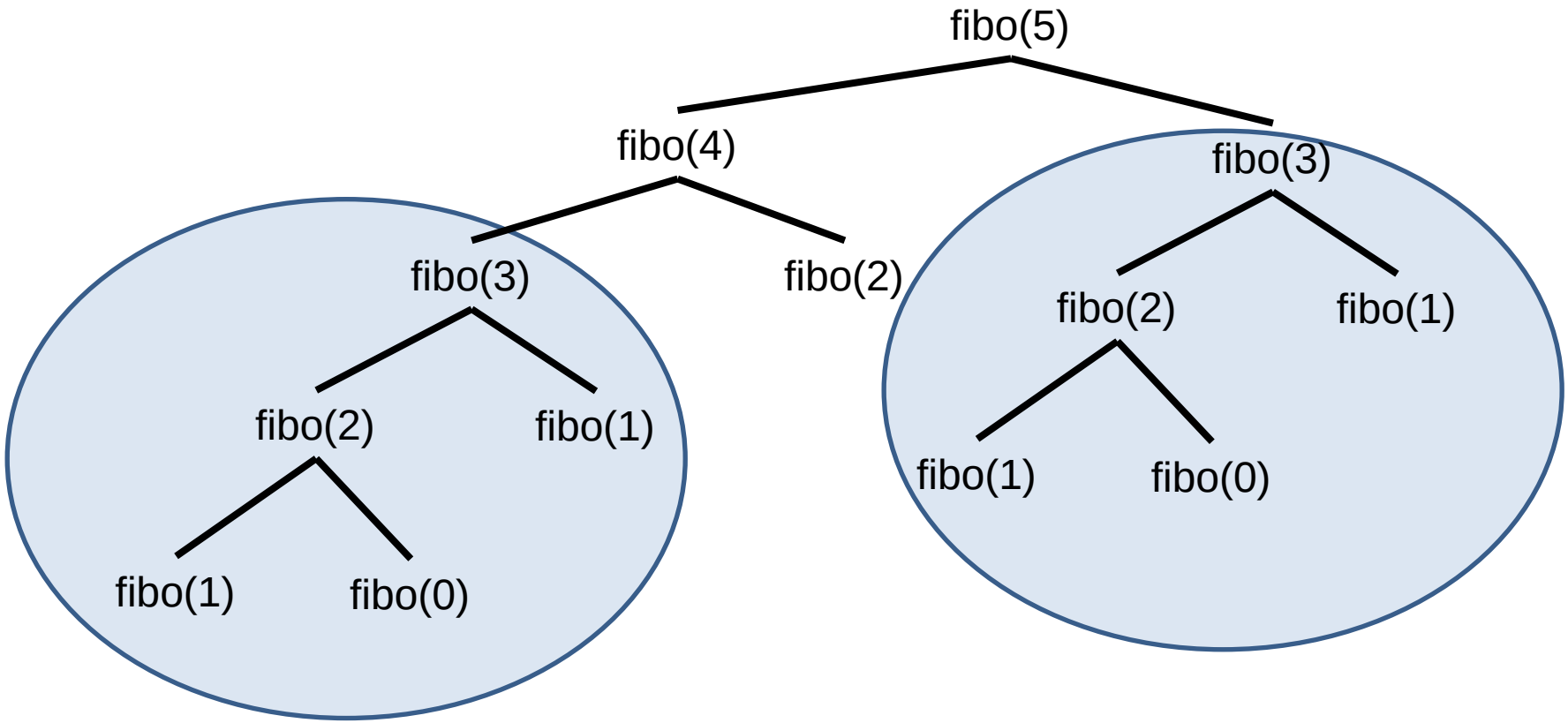
- ตัวอย่างเช่น binary search ไม่มีปัญหาย่อยที่ซ้ำกัน
- binarysearch(68)



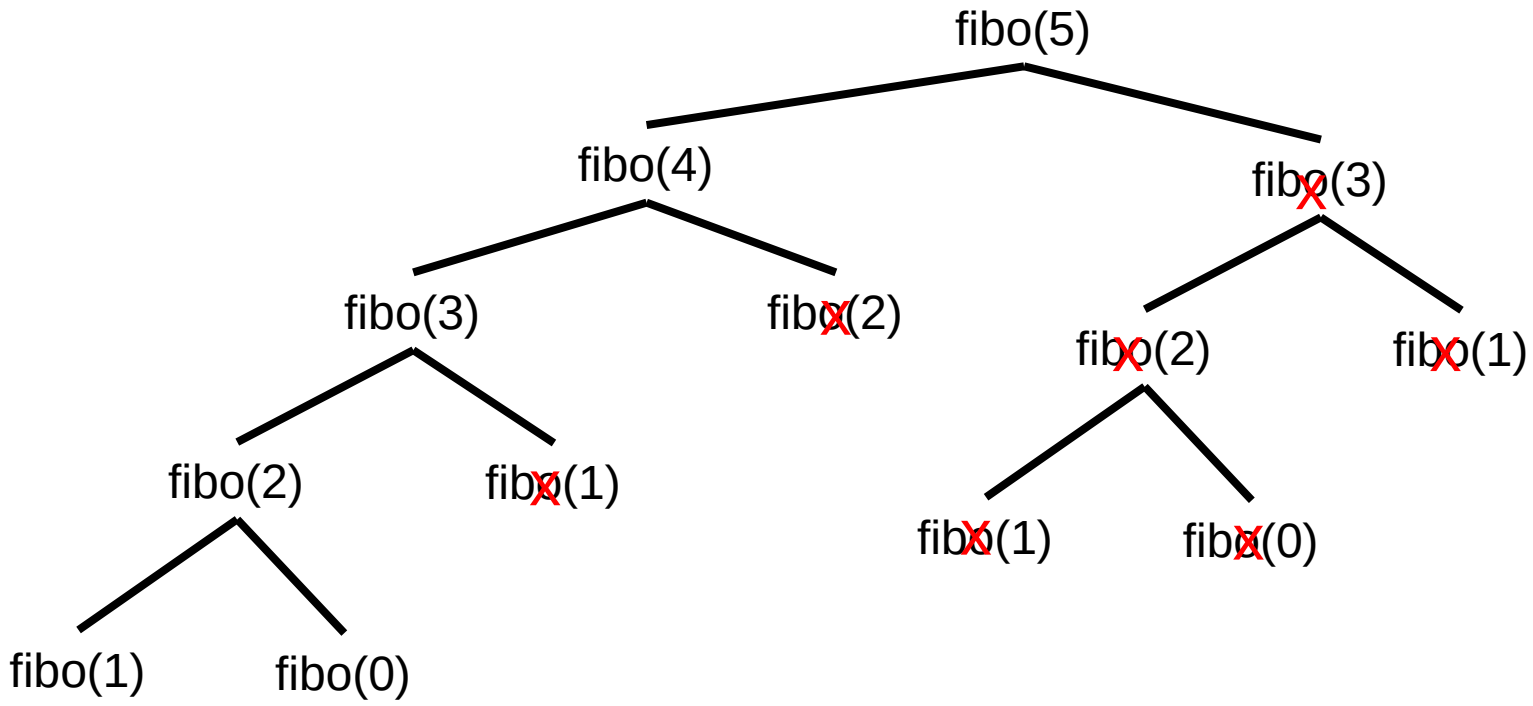
Fibonacci number

- Fibonacci number เป็นตัวอย่างของปัญหาที่มีการเรียกหาคำตอบของปัญหาย่อยซ้ำๆ กันมากๆ

```
int fibo(int n){
    if(n==0 || n==1)
        return 1;
    return fibo(n-1)+fibo(n-2);
}
```



- เห็นได้ว่า fibo(3) ถูกเรียก 2 ครั้ง ถ้าเราเก็บค่าของ fibo(3) แทนที่จะคำนวณใหม่อีกรอบ เราก็เอาค่าที่เก็บไว้มาใช้ได้เลย



- เห็นได้ว่า fibo(3) ถูกเรียก 2 ครั้ง ถ้าเราเก็บค่าของ fibo(3) แทนที่จะคำนวณใหม่อีกรอบ เราก็เอาค่าที่เก็บไว้มาใช้ได้เลย

- วิธีในการเก็บค่าเพื่อนำมาใช้ใหม่มีหลักๆ 2 วิธี
  - Memoization (Top down)
  - Tabulation (Bottom up)

Memoization (Top down)

- การจำคำตอบในลักษณะนี้คล้ายกับการเขียนแบบ Recursive ที่มีการปรับปรุงเล็กน้อย
- วิธีนี้จะ**มองหาคำตอบใน Lookup Table ก่อน**ที่จะคำนวณคำตอบ
- เราจะเริ่มด้วยการกำหนดค่า Lookup table ด้วยค่าเริ่มต้นเป็น NULL ก่อน เมื่อไรก็ตามที่เราต้องการคำตอบของปัญหาย่อย เริ่มต้นเราจะ**ค้นหาใน Lookup table ก่อน**
  - ถ้ามีค่าที่คำนวณไว้แล้วเราก็จะนำมาใช้เลย
  - แต่ถ้าไม่มีเราก็จะคำนวณค่าแล้วเก็บไว้ใน Lookup Table เพื่อไว้ใช้ในคราวต่อไป

ตัวอย่าง Fibonacci number

```
int lookup[20];
void init(){
        int i;
        for(i=0;i<20;i++){
                lookup[i]=NULL;
        }
}
int fibo(int n){
        if(lookup[n] == NULL){
                if(n<=1){
                        lookup[n] = n;
                }else{
                        lookup[n] = fibo(n-1)+fibo(n-2);
                }
        }
        return lookup[n];
}
```

สอบถามใน lookup ก่อน

Tabulation (Bottom up)

- การ**สร้างตาราง**นั้นจะสร้างจาก**ล่างขึ้นบน** (จากตัวแรกไปตัวท้าย จากส่วนเล็กสุดสร้างคำตอบขึ้นให้ส่วนบนสุด) จากนั้นจะคืนค่าในช่องสุดท้ายจากตารางเป็นคำตอบ

- หากใช้ในตัวอย่าง Fibonacci number เราก็จะสร้างตารางคำนวณแล้วเก็บค่า fibo(0) จากนั้นคำนวณแล้วเก็บค่า fibo(1) จากนั้นคำนวณแล้วเก็บค่า fibo(2) จากนั้นคำนวณแล้วเก็บค่า fibo(3) ไปเรื่อยๆ

```
int fibo(int n){
      int f[n+1];
      int i;
      f[0] = 0;
      f[1] = 1;
      for(i = 2; i <= n; i++){
            f[i] = f[i-1] + f[i-2];
      }
      return f[n];
}
```

ทั้งแบบ Tabulation และ Memoization นั้นจะเก็บคำตอบของปัญหาย่อย

- ในแบบ Memoization นั้นตารางจะถูกเติมตามคำสั่งคือ**ทำงานเมื่อถูกเรียกให้สร้างคำตอบเท่านั้น**
- ขณะที่แบบ Tabulation เริ่มจากช่องแรก จากนั้น**ทุกช่องจะค่อยๆ ถูกเติม**ลงไป
- นั่นคือแบบ Memoization ทุกช่องใน Lookup table อาจจะไม่ถูกใส่ข้อมูล
- ทั้งนี้หากลอง Recursive เทียบกับ Tabulation และ Memoization จะพบว่าใช้เวลามากกว่ามากๆๆ (ลองหา Fibonacci number ค่ามากๆ ได้)

เปรียบเทียบ Top-Down กับ Bottom-Up

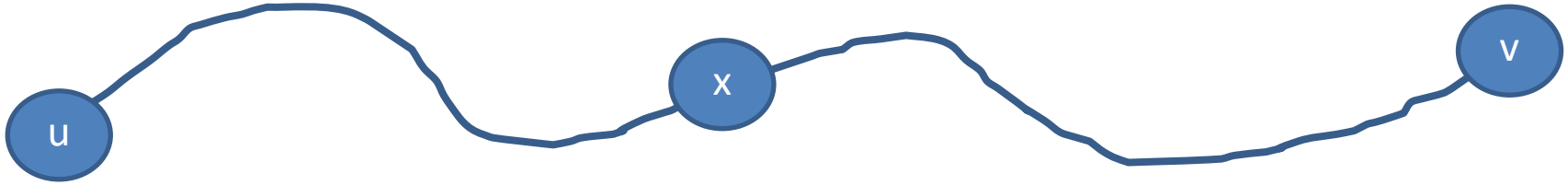
Top-Down	Bottom-Up
ข้อดี - เป็นการเปลี่ยนมาจาก Complete search แบบ Recursion - คำนวณปัญหาย่อยเมื่อจำเป็น (บางครั้งจึงเร็วกว่า)	**ข้อดี** - เร็วกว่าถ้า sub-problem ถูกเรียกบ่อยๆ เพราะว่าไม่มี overhead จาก recursive call - ประหยัดหน่วยความจำ
ข้อเสีย - ช้ากว่าถ้า sub-problem ถูกเรียกบ่อยๆ เพราะว่า overhead ของ function call (ส่วนใหญ่ในการแข่งเขียนโปรแกรมก็ผ่านอยู่ดี)	**ข้อเสีย** - เขียนยากกว่าเพราะว่าไม่ได้แปลงจาก recursive ตรงๆ

Optimal substructure

ปัญหาจะมี Optimal substructure ถ้า**คำตอบที่ดีที่สุด (Optimal solution)** ของปัญหานั้นสามารถ**หา**คำตอบได้จากการใช้**คำตอบที่ดีที่สุดของปัญหาย่อย**ของมัน

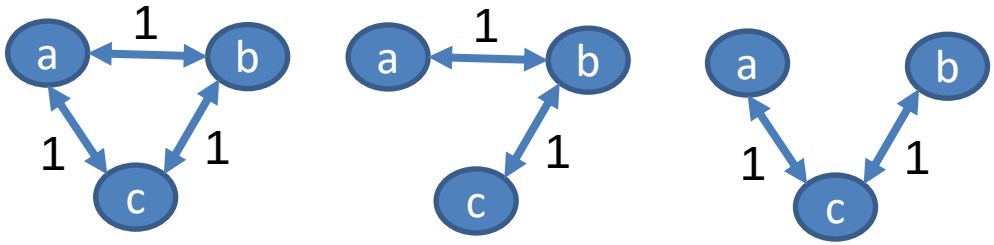
<u>ตัวอย่างเช่น</u> ปัญหา Shortest path

ถ้าโหนด x อยู่ใน shortest path จากโหนด(Source) u ไปโหนด v แล้ว shortest path จาก u ไป v คือการรวมกันของ shortest path จาก u ไป x แล้วจาก x ไป v



แต่ในทางตรงข้ามปัญหา Longest path นั้น**ไม่มี** Optimal substructure

ตัวอย่างเช่นระยะทางยาวที่สุดจาก a ไป c นั่นคือ a–>b–>c แต่ระยะทางที่ยาวที่สุดจาก a ไป b คือ a–>c–>b



นั่นคือ ระยะทางยาวที่สุดจาก a ไป c ไม่ได้เกิดจาก ระยะทางยาวที่สุดจาก a ไป b รวมกับ ระยะทางยาวที่สุดจาก b ไป c

การแก้ Dynamic programming

จริงๆ แล้วมีหลายแนวทางในการแก้

- ขั้นตอนในการแก้ DP

1. Identify if it is a DP problem
2. Decide a state expression with least parameters and Formulate state relationship
3. Do tabulation (or add memoization)

โดยทั่วไปแล้ว ปัญหาเกี่ยวกับการ**หาค่าที่ดีที่สุด**(Optimization problem) ค่าที่มากหรือน้อยที่สุด หรือปัญหาเกี่ยวกับการนับที่บอกว่าให้นับรูปแบบการจัดเรียงภายใต้เงื่อนไขบางอย่าง มักจะแก้ได้ด้วย Dynamic programming

ทั้งนี้ทุกปัญหา Dynamic programming นั้นจะมีคุณสมบัติ Overlapping subproblems (เห็นได้ง่ายกว่า) และปัญหาคลาสสิคส่วนใหญ่ของ Dynamic programming นั้นจะมี Optimal substructure เมื่อเราสังเกตได้ว่ามีคุณสมบัติสองอย่างนี้ในปัญหาจะค่อนข้างมั่นใจได้ว่าแก้ได้ด้วย Dynamic programming

Deciding the state

- ปัญหา DP จะเกี่ยวข้องกับ State และ Transition นั่นคือมีการเปลี่ยนจากสถานะหนึ่งไปเป็นอีกสถานะหนึ่ง

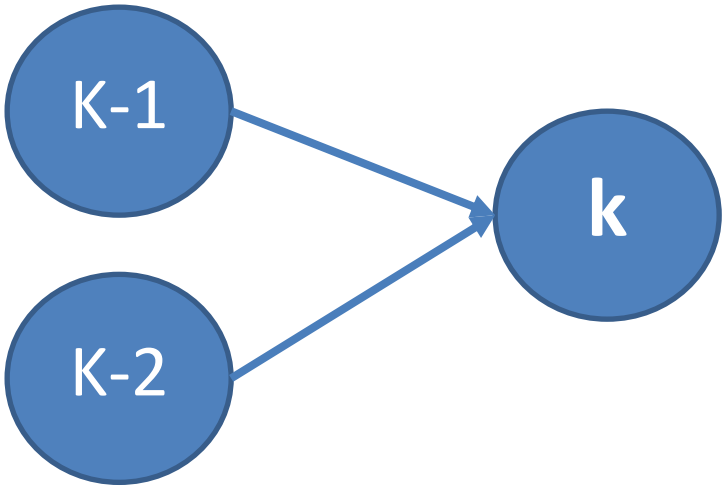
- State สามารถถูกนิยามได้ว่าเป็น**เซตของ parameter** ที่บ่งบอกตำแหน่งเฉพาะในปัญหานั้นได้ เช่นสถานะเมื่อพิจารณาไปแล้ว k ตัว, สถานเมื่อไปถึงโหนดที่ k, พิจารณาของที่มีน้ำหนัก k และขนาดถุงที่ใส่ได้ w

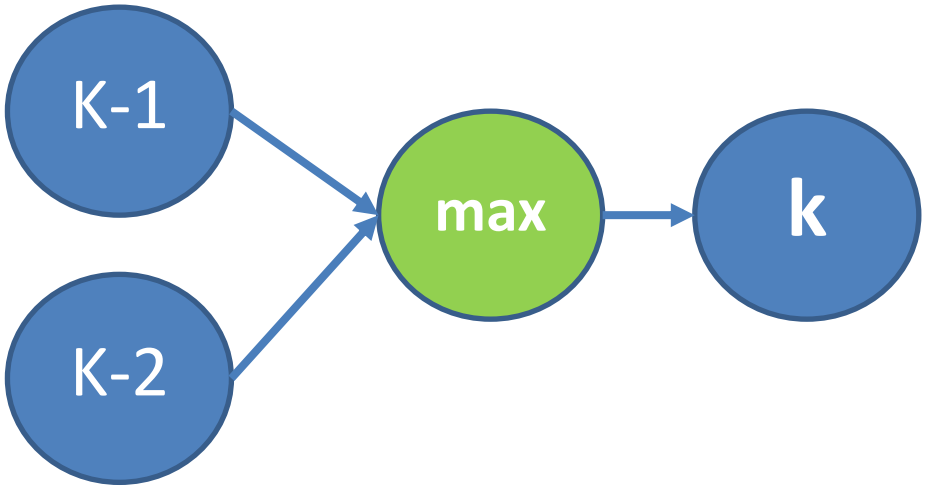
- เซตของ parameter นี้ควรจะมี**จำนวนน้อยที่สุด**เท่าที่จะเป็นไปได้เพื่อที่จะเป็นการลด state space (ลดขนาดตารางนั่นเอง)

Formulating a relation among the states

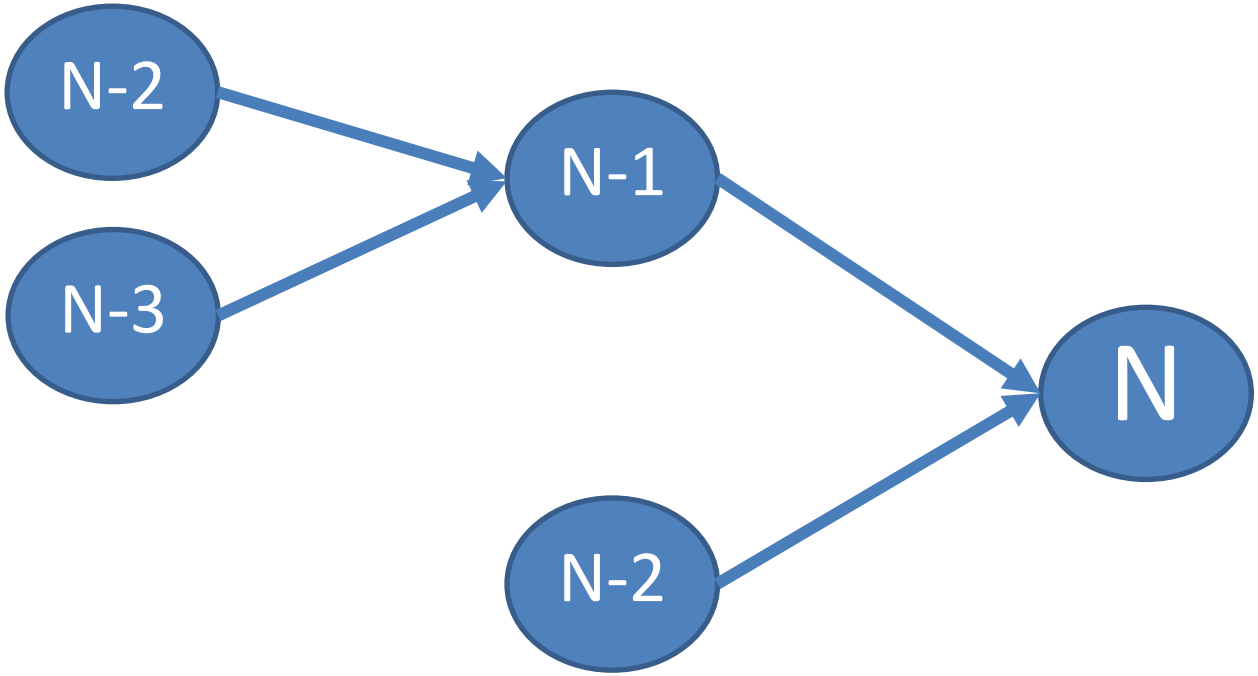
ความสัมพันธ์ระหว่าง state ส่วนนี้เป็นส่วนที่ยากที่สุดของการแก้ DP และต้องการความคิดสร้างสรรค์ การสังเกต และการฝึกฝน

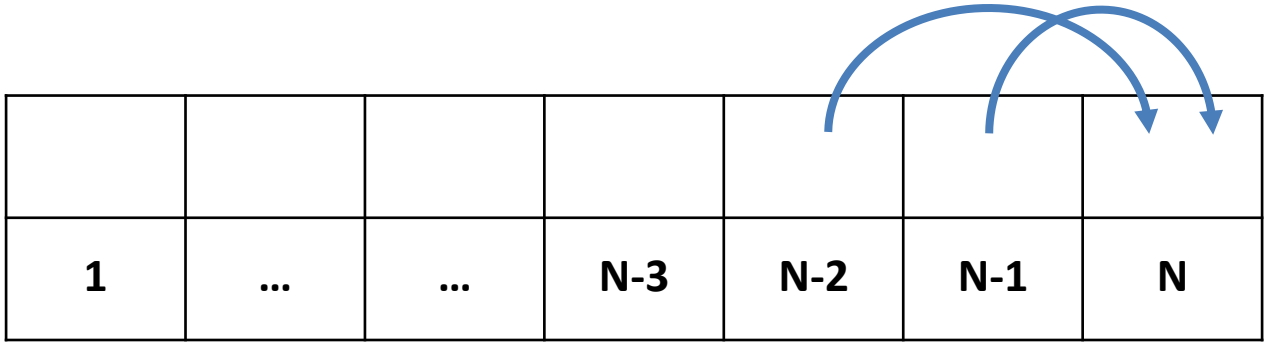
state ปัจจุบัน เกิดจาก state เก่าๆ ก่อนหน้าได้อย่างไร





Fibonacci number





ลองพิจารณาตัวอย่างต่อไปนี้

กำหนดให้ ตัวเลข 3 ตัวเลขได้แก่ 1, 3, 5 จงหาว่ารูปแบบทั้งหมดที่เป็นไปได้ในการรวมกันเป็นจำนวนที่มีค่า N จาก 3 จำนวนนี้

ตัวอย่างเช่นเลข 5

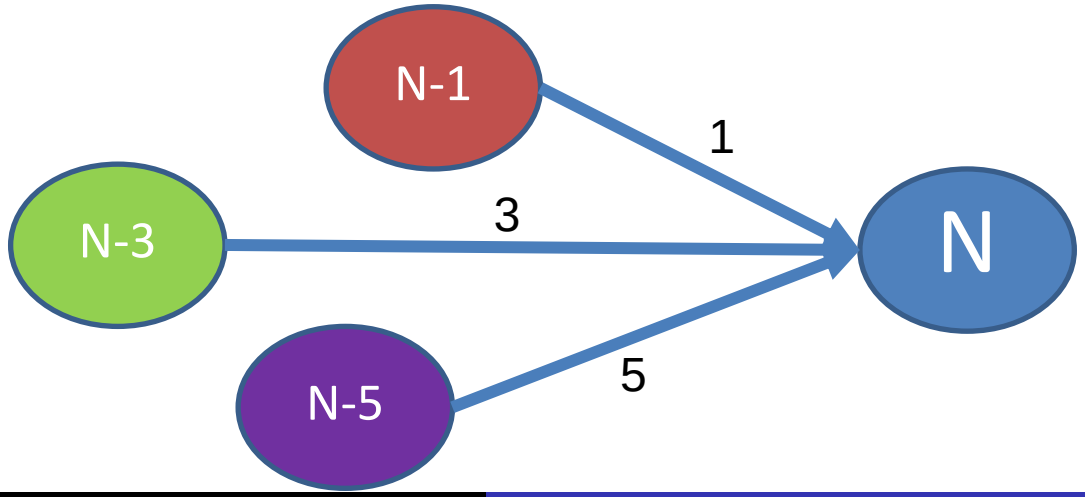
1+1+1+1+1 3+1+1 1+3+1 1+1+3 5

ลองพิจารณาปัญหาข้างต้น

เริ่มต้นเราก็คิด "**state**" ของปัญหาก่อน นั่นคือเราจะนำเอา parameter n มาใช้ เพื่อบอกสถานะว่ามันจะระบุปัญหาย่อยใดๆ ได้อย่างไร

พิจารณา สถานะที่ n, ตำแหน่งที่ n, ค่าที่ n, ชิ้นที่ n มีลักษณะอย่างไร

ดังนั้น state ของ dp ก็จะหน้าตาประมาณนี้ state(n) โดยที่ state(n) หมายถึง**จำนวนทั้งหมด**ในการเขียนให้ได้ค่า n โดยใช้ 1, 3, 5



ต่อไปจะเป็นการคำนวณ state(n)

- เนื่องจากเราสามารถใช้ได้เพียง 1, 3, 5 ในการสร้าง n
- สมมติว่าเรารู้ว่าผลลัพธ์ของ n = 1, 2, 3, 4, 5, 6
  นั่นคือ เรารู้ผลลัพธ์ของ state(n=1), state(n=2), state(n=3), ... , state(n=6)
  เมื่อเราต้องการหาค่าของ state(n=7) เราจะทำได้อย่างไรบ้าง

- เราจะเพิ่ม 1, 3, 5 ได้เท่านั้น นั่นคือเราสามารถรวมเป็น 7 ได้เพียง 3 วิธี คือ **ใช้ 1 ต่อท้าย**, **ใช้ 3 ต่อท้าย**, **ใช้ 5 ต่อท้าย**

- หากใช้ 3 คือ **เราเพิ่ม 3 ต่อท้าย** แล้วค่าก่อนหน้าที่เราจะเพิ่มค่า 3 คืออะไร คือค่า (n=4) นั่นเอง ดังนั้น หากเราเพิ่ม 3 ต่อท้ายก็จะมีจำนวนแบบเท่ากับ state(n=4) **นั่นคือ ไปถามว่าเขียน 4 ได้กี่แบบนั่นเอง**

[(1+1+1+1)+3]

[(3+1)+3]

[(1+3)+3]

- หากใช้ 5 คือ **เราเพิ่ม 5 ต่อท้าย** แล้วค่าก่อนหน้าที่เราจะเพิ่มค่า 5 คืออะไร คือค่า (n=2) นั่นเอง ดังนั้น หากเราเพิ่ม 5 ต่อท้ายก็จะมีจำนวนแบบเท่ากับ state(n=2) **นั่นคือ ไปถามว่าเขียน 2 ได้กี่แบบนั่นเอง**

[(1+1)+5]

- หากใช้ 1 คือ **เราเพิ่ม 1 ต่อท้าย** แล้วค่าก่อนหน้าที่เราจะเพิ่มค่า 1 คืออะไร คือค่า (n=6) นั่นเอง ดังนั้น หากเราเพิ่ม 1 ต่อท้ายก็จะมีจำนวนแบบเท่ากับ state(n=6) นั่นคือ **ไปถามว่าเขียน 6 ได้กี่แบบนั่นเอง**

[(1+1+1+1+1+1)+1]

[(1+1+1+3)+1]

[(1+1+3+1)+1]

[(1+3+1+1)+1]

[(3+1+1+1)+1]

[(3+3)+1]

[(1+5)+1]

[(5+1)+1]

- ต่อไปลองคิดว่าทั้งสามกรณีก่อนหน้านี้ครบทุกกรณีแล้วหรือยังในการรวมกันให้ได้ 7
- ดังนั้นเราสามารถบอกว่าผลลัพธ์ของ

state(7) = state(7–1) + state(7–3) + state(7–5)

ในรูปแบบทั่วไปคือ

state(n) = state(n–1) + state(n–3) + state(n–5)

แล้ว case ง่าย

- เราย้อนกลับมาเรื่อยๆ คำถามคือ ย้อนกลับมาถึงเมื่อไร
- ทำให้สิ่งสำคัญของ DP คือการใช้ recursive ด้วย

- ถ้าเป็น 0 ตอบ 1 วิธี
- ถ้าติดลบ เป็น 0 วิธี

```
int solve(int n)
{
    // base case
    if (n < 0)
        return 0;
    if (n == 0)
        return 1;

    return solve(n-1) + solve(n-3) + solve(n-5);
}
```

จากตัวอย่างข้างบนจะใช้เวลาในการทำงานนานมากเป็น exponential time ดังนั้นต่อไปเราจะมาเพิ่มส่วน memoization

Adding memorization or tabulation for the state

- นี่เป็นส่วนที่ง่ายสุดของ dp เราเพียงแค่**เก็บ** state ของคำตอบ เพื่อที่ว่าในการสอบถามคราวหน้า เราจะเอาคำตอบมาใช้ได้เลย โดยการเพิ่ม code

```
int dp[MAXN];
int solve(int n)
{
  // base case
  if (n < 0)
      return 0;
  if (n == 0)
      return 1;
  // checking if already calculated
  if (dp[n]!=0)
      return dp[n];
  // storing the result and returning
  return dp[n] = solve(n-1) + solve(n-3) + solve(n-5);
}
```

```
int dp[MAXN];
int solve(int n)
{  // base case
  dp[0] = 1;
  dp[1] = 1;
  dp[2] = 1;
  dp[3] = 2;
  dp[4] = 3;
  for(int i=5; i<=n; i++)
      dp[i] = dp[i-1] + dp[i-3] + dp[i-5];
  return dp[n];
}
```

Binomial coefficient calculator

- Binomial coefficient C(n, k) นิยามได้เป็นสัมประสิทธิของ x^k เมื่อกระจาย $(1 + x)^n$.

- ทั้งนี้ binomial coefficient C(n, k) ให้จำนวนวิธีที่ของ k ชิ้นจะถูกเลือกจากของ n ชิ้น

ในข้อนี้

- จงเขียนฟังก์ชันที่รับ parameters n และ k จากนั้นคืนค่าของ Binomial Coefficient C(n, k).

ตัวอย่างเช่น

- หากฟังก์ชันรับค่า n = 4 and k = 2 จะคืนค่า 6

- หากฟังก์ชันรับค่า n = 5 and k = 2 จะคืนค่า 10

- เมื่อพูดถึง dynamic programming จะมี องค์ประกอบ 2 ส่วนที่
  - optimal substructure
    - (ปัญหาใหญ่สร้างได้มาจากปัญหาเล็ก recursive นั่นเอง)
  - overlapping subproblem
    - (เรียกปัญหาเล็กซ้ำๆ)

Optimal Substructure

- ค่าของ C(n, k) สามารถคำนวณแบบ recursive ได้โดยใช้สูตรมาตรฐานของ Binomial Coefficients นั่นคือ

- $C(n, k) = C(n-1, k-1) + C(n-1, k)$
- $C(n, 0) = C(n, n) = 1$

```
int binomialCoeff(int n, int k){
    // Base Cases
    if (k > n)
        return 0;
    if (k == 0 || k == n)
        return 1;
    // Recursive Cases
    return binomialCoeff(n - 1, k - 1)+ binomialCoeff(n - 1, k);
}
```

Overlapping Subproblems

สังเกตได้ว่า ฟังก์ชันนี้มี subproblems ที่เหมือนกันหลายจุด

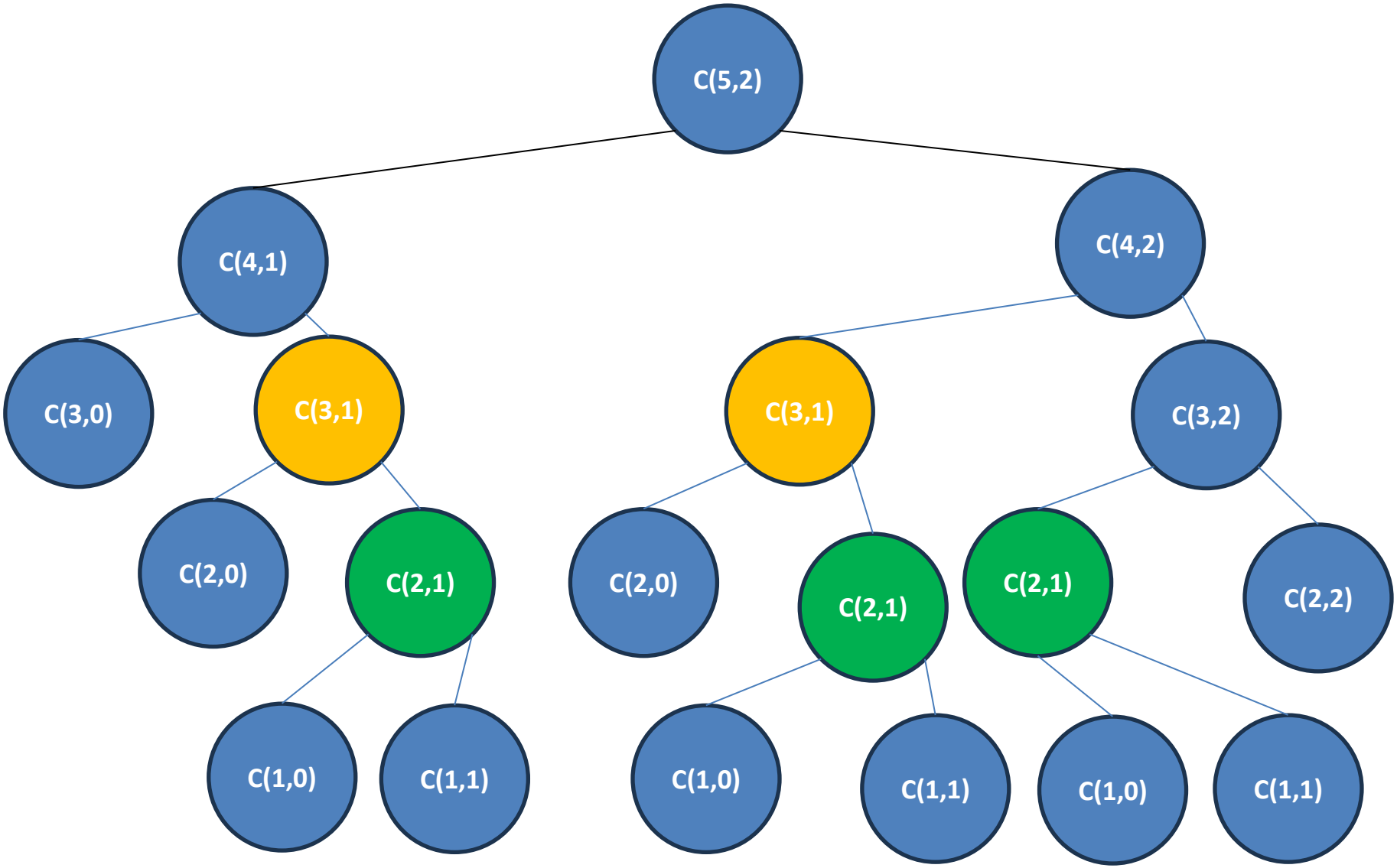
ลองพิจารณาต้นไม้ recursion เมื่อ n = 5 and k = 2.

สังเกตว่า C(3, 1) ถูกเรียนก 2 ครั้ง

แน่นอนสำหรับค่า n ที่มากขึ้น จะมีการซ้ำของ subproblems ที่มากขึ้น

ดังนั้นทำอย่างไรดี

C(5,2)


C(5,2)
C(4,1)
C(4,2)
C(3,0)
C(3,1)
C(3,1)
C(3,2)
C(2,0)
C(2,1)
C(2,0)
C(2,1)
C(2,1)
C(2,2)
C(1,0)
C(1,1)
C(1,0)
C(1,1)
C(1,0)
C(1,1)

```
int binomialCoeff (int n, int k){
    if (dp[n][k] != -1) //
        return dp[n][k];
    // store value in a table before return
    if (k == 0) {
        dp[n][k] = 1;
        return dp[n][k];
    }
    // store value in table before return
    if (k == n) {
        dp[n][k] = 1;
        return dp[n][k];
    }
    // save value in lookup table before return
    dp[n][k] = binomialCoeff (n - 1, k - 1) +
               binomialCoeff (n - 1, k);
    return dp[n][k];
}
```

```
int binomialCoeff(int n, int k)
{
    int C[n + 1][k + 1], i, j;
    for (i = 0; i <= n; i++) {
        for (j = 0; j <= min(i, k); j++) {
            // Base Cases
            if (j == 0 || j == i)
                C[i][j] = 1;

            // Calculate value using previously
            // stored values
            else
                C[i][j] = C[i - 1][j - 1] + C[i - 1][j];
        }
    }
    return C[n][k];
}
```